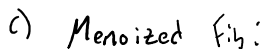


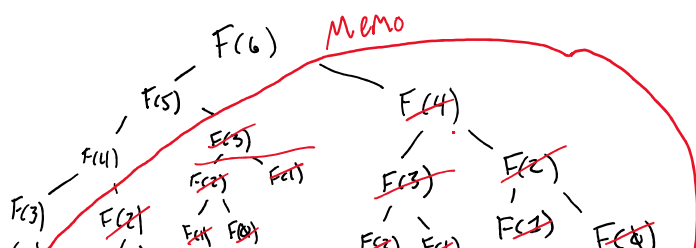
- a) Recursive Fib:

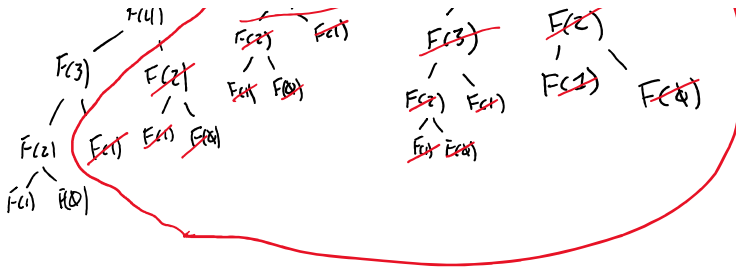
b)



```
int MemoizedFibAux(int fibNum, int Memo[]) {
    if (fibNum == 0) {
        return 0;
    }
    if (fibNum == 1) {
        return 1;
    }
}
```

d)





e)

```

int bottomUP Fibonacci(int fibNum)
{
    if (fibNum <= 0) {
        return 0;
    }
    else if (fibNum == 1) {
        return 1;
    }
    int lookupArraySize = fibNum + 1; // +1 instead of -1 because need to store 0
    int lookupArray[lookupArraySize];
    lookupArray[0] = 0;
    lookupArray[1] = 1;
    for (int i = 2; i <= fibNum; i++) {
        lookupArray[i] = lookupArray[i-1] + lookupArray[i-2];
    }
    return lookupArray[fibNum];
}
  
```

2. Starting w/ standard solution in C++...

```

BOTTOM-UP-CUT-ROD(p, N)
{
    int r[N];
    r[0] = 0;
    for (int j = 1; j <= N; j++) {
        int q = -∞;
        for (int i = 1; i <= j; i++) {
            q = max(q, price[i] + r[j-i]);
        }
        r[j] = q;
    }
    return r[N];
}
  
```

MODIFIED-CUT-ROD(int price[], int n, int c)

```

int r[N];
r[0] = 0;
for (int j = 1; j <= n; j++) {
    int q = -∞;
    for (int i = 1; i <= j; i++) {
        q = max(q, price[i] + r[j-i] - c);
    }
}
  
```

Would just need
to factor in cost
per level
↓

```

    }
    r[j] = q;
}
return r[n]
}

```

3. Starting w/ Regular Memoized cut Rod in c++:

```

int MEMOIZED-CUT-ROD (int price[], int n) {
    int r[];
    for (i = 0, i < n, i++) {
        r[i] = -∞;
    }
    return MEMOIZED-CUT-ROD-AUX (price, n, r);
}

int MEMOIZED-CUT-ROD-AUX (int price[], int n, int r[]) {
    int q;
    if (r[n] >= 0) {
        return r[n];
    }

    if (n == 0) {
        q = 0;
    }
    else {
        q = -∞;
        for (int i = 1, i < n, i++) {
            q = MAX {q, price[i-1] + MEMOIZED-CUT-ROD-AUX (price, n-i, r)};
        }
    }
    r[n] = q;
    return q;
}

```

```

MEMOIZED-CUT-ROD (int price[], int n) {
    int r[];
    for (i = 0, i < n, i++) {
        r[i] = -∞;
        s[i] = -∞;
    }

    return MEMOIZED-CUT-ROD-AUX (price, n, r, s);
}

MEMOIZED-CUT-ROD-AUX (int price[], int n, int r[], int s[]) {
    int q;
    if (r[n] >= 0) {
        return r[n];
    }

    if (n == 0) {
        q = 0;
    }
    else {
        q = -∞;
    }
}

```

```

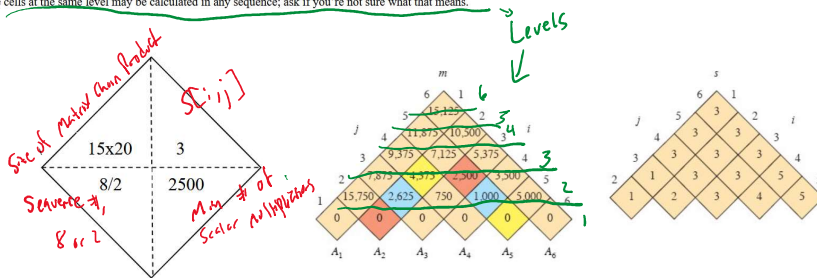
q = -∞;
for (int i = 1; i < n; i++) {
    int temp = MAX { Price[i] + MINIMIZE CUT-ROD-AUX (Price, n-i, r);
    if (temp > q) {
        q = temp;
        s[n] = i;
    }
}
}
r[n] = q;
return (q, s);

```

4. [10 pts.] Find the optimal parenthesization for the matrices A_1, A_2, A_3, A_4 , and A_5 , which are respectively of sizes $2 \times 3, 3 \times 10, 10 \times 5, 5 \times 4$, and 4×20 . You must show your work using a diagram similar to Figure 14.5 on page 380, which clearly shows $m[i,j]$ and $s[i,j]$ for all combinations that apply. However, I have found a variant of that diagram very convenient, and so I recommend that you show FOUR items in each of the cells of the diagram for position $[i,j]$ that combine (as well as augment) information from both parts of Figure 14.5, as I have shown in my illustration below. ALSO show your work in arriving at the answer, i.e., how you determine the minimum number of scalar multiplications, etc. by considering multiple alternative options.

Each of your cells may have to be a bit bigger to be able to show four items, one in each quadrant (if you are handwriting, you may consider writing the two wider items, e.g., 15×20 and 2500 , slanted a bit diagonally). Each cell includes the size of the matrix chain product $A_i \dots A_j$, e.g., 15×20 in the top left quadrant; the $s[i,j]$ value, e.g., 3, in the top right quadrant; the minimal number of scalar multiplications $m[i,j]$ in the matrix chain product $A_i \dots A_j$, e.g., 2500, in the bottom right quadrant; and the sequence number, i.e., the sequence/level* in which this cell's contents are calculated, e.g., 8 or 2, in the bottom left quadrant. You may find it convenient to use Matrix-Chain-Order on page 378 (or perhaps less intuitively, the memorized version described later on page 391) in doing this. Note that when I equalize items for the original matrices, only the matrix size is relevant, since the number of scalar multiplications would be 0, and $s[i,j]$ and sequence number don't really apply.

*NOTE: The cells at the same level may be calculated in any sequence; ask if you're not sure what that means.



A_1 A_2 A_3 A_4 A_5
 2×3 3×10 10×5 5×4 4×20

$M[1,1]$

A_1

$M[1,2]$

A_1, A_2

$2 \times 3 \times 10 = 60$

$M[1,3]$

$A_1 \cdot (A_2 \cdot A_3)$

$2 \times 3 \times 10 = 60$
 $M[1,1] + M[2,3] + 5 \times 4 \times 2$
 $0 + 60 + 40 = 100$

$M[2,3]$

A_2, A_3

$3 \times 10 \times 5 = 150$

$M[3,4]$

$A_3 \cdot A_4$

$10 \times 5 \times 4 = 200$

$M[4,5]$

$A_4 \cdot A_5$

$5 \times 4 \times 20 = 400$

	1	2	3	4	5
1	0	60	160		
2		0	150		
3			0	200	
4				0	400
5					0

	1	2	3	4	5
1			2		
2					
3					
4					
5					



1 2 3 4 5

	1	2	3	4	5
1	0	60	160	260	360
2		0	150	250	350
3			0	200	300
4				0	400
5					0

Converted code from book to C++, added macros

1	0	2	6a	3	16a	4	20a	3	36a
2	240	2	3x5	2	3x4	3	3x3	4	3x2
	1	0	2	15a	3	21a	4	45a	
3	1x5	0	1x4	3	1x3	4	1x2		
	1	0	2	2x4	3	1x3			
4	5x4	0	5x3	4	1				
	1	0	2	4x3					
5	4x2	0	1	0					
	1	0							